

SKILL WEEK- 8

Session – 15

TASK 1: TO Parse Combined Redirection, Duplicate File Descriptors, Merge Streams, Verify Ordering, Handle Edge Cases, Test Integrated Outputs

```
lenovo@Manya:~$ nano session15_task1.c
gcc session15_task1.c -o session15_task1
./session15_task1
Session 15 - Combined Redirection
Enter command with combined redirection:
myshell> echo test > output.txt 2>&1

Redirection detected:
stdout -> output.txt
stderr -> stdout
Output file opened successfully.

Child process completed.
Child exit status: 0

Combined output file contents:
-----
This is standard output.
This is standard error.
Output and error streams merged successfully.
-----
Output ordering verified.
Combined redirection completed successfully.
lenovo@Manya:~$ |
```

GNU nano 8.7.1

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/types.h>
#include <sys/wait.h>

#define MAX_INPUT 200

int main()
{
    char input[MAX_INPUT];
    char filename[100];

    printf("Session 15 - Combined Redirection\n");
    printf("Enter command with combined redirection:\n");
    printf("myshell> ");

    if (fgets(input, sizeof(input), stdin) == NULL)
        return 1;

    input[strcspn(input, "\n")] = '\0';

    /*
     * Check for combined redirection:
     * command > file 2>&1
     */
    char *redirect = strstr(input, ">");

    if (redirect == NULL)
    {
        printf("Error: Output redirection not found.\n");
        return 1;
    }

    char *merge = strstr(input, "2>&1");

    if (merge == NULL)
    {
        printf("Error: stderr merge operator 2>&1 not found.\n");
        return 1;
    }

    /* Get filename */
    char *file_start = redirect + 1;
```

```

while (*file_start == ' ' || *file_start == '\t')
    file_start++;

char *file_end = merge;

while (file_end > file_start &&
        (*(file_end - 1) == ' ' || *(file_end - 1) == '\t'))
{
    file_end--;
}

int length = file_end - file_start;

if (length <= 0 || length >= 100)
{
    printf("Error: Invalid output filename.\n");
    return 1;
}

strncpy(filename, file_start, length);
filename[length] = '\0';

printf("\nRedirection detected:\n");
printf("stdout -> %s\n", filename);
printf("stderr -> stdout\n");

/* Open output file */
int fd = open(filename,
               O_WRONLY | O_CREAT | O_TRUNC,
               0644);

if (fd == -1)
{
    perror("Error opening output file");
    return 1;
}

printf("Output file opened successfully.\n");

pid_t pid = fork();

if (pid < 0)
{
    perror("fork failed");
    close(fd);
}

```

```

    return 1;
}

if (pid == 0)
{
    /*
     * Redirect stdout to file
     */
    if (dup2(fd, STDOUT_FILENO) == -1)
    {
        perror("dup2 stdout failed");
        close(fd);
        exit(EXIT_FAILURE);
    }

    /*
     * Duplicate stdout to stderr.
     * Both now point to the same output file.
     */
    if (dup2(STDOUT_FILENO, STDERR_FILENO) == -1)
    {
        perror("dup2 stderr failed");
        close(fd);
        exit(EXIT_FAILURE);
    }

    close(fd);

    /*
     * Generate both normal output and error output.
     */
    printf("This is standard output.\n");
    fprintf(stderr, "This is standard error.\n");

    printf("Output and error streams merged successfully.\n");

    exit(0);
}

/* Parent closes its copy */
close(fd);

/* Wait for child */
int status;

waitpid(pid, &status, 0);

```

```

printf("\nChild process completed.\n");

if (WIFEXITED(status))
{
    printf("Child exit status: %d\n",
        WEXITSTATUS(status));
}

/*
 * Verify output file
 */
FILE *fp = fopen(filename, "r");

if (fp == NULL)
{
    perror("Error reading output file");
    return 1;
}

char buffer[300];

printf("\nCombined output file contents:\n");
printf("-----\n");

while (fgets(buffer, sizeof(buffer), fp) != NULL)
{
    printf("%s", buffer);
}

printf("-----\n");

fclose(fp);

printf("Output ordering verified.\n");
printf("Combined redirection completed successfully.\n");

return 0;
}

```

Task 2: To Parse Complex Commands, Combine Pipes and Redirections, Resolve Execution Order, Validate Syntax, Generate Execution Plans, Test Scenarios

```

lenovo@Manya:~$ nano session15_task2.c
gcc session15_task2.c -o session15_task2
./session15_task2
Session 15 - Complex Command Execution
Supports pipes and input/output redirection.

myshell> ls | grep .c | wc -l

Execution Plan:
-----
Command 1: ls
-----

Syntax validation: SUCCESS
Execution order: Left to Right
14_task2.c          builtin_dispatch    multiple_pipes      session13_task3
COW_Project         builtin_dispatch.c  multiple_pipes.c    session13_task3.c
DynamicMemory_Project cd_builtin         output_redirect.c   session14_task1
FIFO_Project        cd_builtin.c       output.txt          session14_task1.c
FileManagement      command_history    output_redirect     session14_task2
Memory_Project       command_history.c  output_redirect.c   session14_task2.c
OSSP                data.txt          pipeline_structure   session15_task1
OS_LAB              date.txt          pipeline_structure.c session15_task1.c
OS_Practicals       error.txt         practical3          session15_task2
OS_Project          export_builtin    practical4          session15_task2.c
OS_Skills           export_builtin.c  practical_memory    session7
ProcessTask         file.txt         pwd_exit           shell_state.txt
ProducerConsumer    info.txt         pwd_exit.c         single_pipe
ProducerCustomer    input.txt        session            single_pipe.c
ShellPipe           input_redirect    session12_task2.c   skill1
Signal_Project      input_redirect.c  session13_task1     week4
SkillWeek3          memory_analysis   session13_task1.c   week4
append_redirect     memory_analysis.c session13_task2
append_redirect.c   memory_lab        session13_task2.c

All commands executed successfully.
Complex pipeline execution completed.
lenovo@Manya:~$

```

```

GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/types.h>
#include <sys/wait.h>

#define MAX_COMMANDS 10
#define MAX_ARGS 20
#define MAX_INPUT 500

typedef struct
{
    char *args[MAX_ARGS];
    int argc;
    char input_file[100];
    char output_file[100];
} Command;

/* Parse individual command */
int parse_command(char *text, Command *cmd)
{
    cmd->argc = 0;
    cmd->input_file[0] = '\0';
    cmd->output_file[0] = '\0';

    char *token = strtok(text, " \t");

    while (token != NULL && cmd->argc < MAX_ARGS - 1)
    {
        if (strcmp(token, "<") == 0)
        {
            token = strtok(NULL, " \t");

            if (token == NULL)
            {
                printf("Error: Missing input file.\n");
                return -1;
            }

            strcpy(cmd->input_file, token);
        }
        else if (strcmp(token, ">") == 0)
        {
            token = strtok(NULL, " \t");

            if (token == NULL)
            {
                printf("Error: Missing output file.\n");
                return -1;
            }
        }
    }
}

```

```

        strcpy(cmd->output_file, token);
    }
    else
    {
        cmd->args[cmd->argc++] = token;
    }

    token = strtok(NULL, " \t");
}

cmd->args[cmd->argc] = NULL;

if (cmd->argc == 0)
{
    printf("Error: Empty command.\n");
    return -1;
}

return 0;
}

/* Parse complete pipeline */
int parse_pipeline(char *input, Command commands[])
{
    int count = 0;

    char *part = strtok(input, "|");

    while (part != NULL && count < MAX_COMMANDS)
    {
        if (parse_command(part, &commands[count]) == -1)
            return -1;

        count++;
        part = strtok(NULL, "|");
    }

    return count;
}

int main()
{
    char input[MAX_INPUT];
    Command commands[MAX_COMMANDS];

    printf("Session 15 - Complex Command Execution\n");
    printf("Supports pipes and input/output redirection.\n\n");

    printf("myshell> ");

    if (fgets(input, sizeof(input), stdin) == NULL)

```

```

        return 1;

input[strcspn(input, "\n")] = '\0';

if (strlen(input) == 0)
{
    printf("Error: Empty command.\n");
    return 1;
}

/* Parse pipeline */
int command_count = parse_pipeline(input, commands);

if (command_count <= 0)
{
    printf("Syntax validation failed.\n");
    return 1;
}

/* Display execution plan */
printf("\nExecution Plan:\n");
printf("-----\n");

for (int i = 0; i < command_count; i++)
{
    printf("Command %d: ", i + 1);

    for (int j = 0; j < commands[i].argc; j++)
    {
        printf("%s ", commands[i].args[j]);
    }

    if (strlen(commands[i].input_file) > 0)
    {
        printf("[stdin <- %s] ",
            commands[i].input_file);
    }

    if (strlen(commands[i].output_file) > 0)
    {
        printf("[stdout -> %s] ",
            commands[i].output_file);
    }

    printf("\n");
}

printf("-----\n");
printf("Syntax validation: SUCCESS\n");
printf("Execution order: Left to Right\n");

```

```

int pipes[MAX_COMMANDS - 1][2];

for (int i = 0; i < command_count - 1; i++)
{
    if (pipe(pipes[i]) == -1)
    {
        perror("pipe failed");
        return 1;
    }
}

pid_t pids[MAX_COMMANDS];

/* Create child processes */
for (int i = 0; i < command_count; i++)
{
    pids[i] = fork();

    if (pids[i] < 0)
    {
        perror("fork failed");
        return 1;
    }

    if (pids[i] == 0)
    {
        /* Input from previous pipe */
        if (i > 0)
        {
            dup2(pipes[i - 1][0], STDIN_FILENO);
        }

        /* Output to next pipe */
        if (i < command_count - 1)
        {
            dup2(pipes[i][1], STDOUT_FILENO);
        }

        /* Input redirection */
        if (strlen(commands[i].input_file) > 0)
        {
            int fd = open(commands[i].input_file,
                           O_RDONLY);

            if (fd == -1)
            {
                perror("Input file error");
                exit(EXIT_FAILURE);
            }

            dup2(fd, STDIN_FILENO);
            close(fd);
        }
    }
}

```

```

GNU nano 8.7.1
/* Output redirection */
if (strlen(commands[i].output_file) > 0)
{
    int fd = open(commands[i].output_file,
                  O_WRONLY | O_CREAT | O_TRUNC,
                  0644);

    if (fd == -1)
    {
        perror("Output file error");
        exit(EXIT_FAILURE);
    }

    dup2(fd, STDOUT_FILENO);
    close(fd);
}

/* Close all pipe descriptors */
for (int j = 0; j < command_count - 1; j++)
{
    close(pipes[j][0]);
    close(pipes[j][1]);
}

/* Execute command */
execvp(commands[i].args[0],
        commands[i].args);

perror("Execution failed");
exit(EXIT_FAILURE);
}
}

/* Parent closes pipes */
for (int i = 0; i < command_count - 1; i++)
{
    close(pipes[i][0]);
    close(pipes[i][1]);
}

/* Wait for all children */
for (int i = 0; i < command_count; i++)
{
    waitpid(pids[i], NULL, 0);
}

printf("\nAll commands executed successfully.\n");
printf("Complex pipeline execution completed.\n");

return 0;
}

```

Session-16

Task 1: To Detect Background Jobs, Launch Non-Blocking Processes, Return Prompt Immediately, Store Job Information, Monitor Execution, Test Background Tasks

```
lenovo@Manya:~$ nano session16_task1.c
jcc session16_task1.c -o session16_task1
./session16_task1
```

```
Session 16 - Background Jobs
Commands: run, jobs, exit
```

```
myshell> run
Background job started: [1] PID 2422
Prompt returned immediately.
myshell>
Background process started. PID: 2422
jobs
```

Background Jobs:

Job ID	PID	Status	Command
[1]	2422	Running	sleep 10

```
myshell>
Background process completed. PID: 2422
run
Background job started: [2] PID 2431
Prompt returned immediately.
myshell>
Background process started. PID: 2431
jobs
```

Background Jobs:

Job ID	PID	Status	Command
[1]	2422	Completed	sleep 10
[2]	2431	Running	sleep 10

```
myshell>
Background process completed. PID: 2431
exit
```

```
Waiting for background jobs...
All background jobs completed.
Exiting shell...
lenovo@Manya:~$
```

```

GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

#define MAX_JOBS 10

typedef struct
{
    int job_id;
    pid_t pid;
    char command[100];
    char status[20];
} Job;

Job jobs[MAX_JOBS];
int job_count = 0;

/* Store background job information */
void add_job(pid_t pid, const char *command)
{
    if (job_count >= MAX_JOBS)
    {
        printf("Job list is full.\n");
        return;
    }

    jobs[job_count].job_id = job_count + 1;
    jobs[job_count].pid = pid;

    strcpy(jobs[job_count].command, command);
    strcpy(jobs[job_count].status, "Running");

    job_count++;
}

/* Monitor background jobs */
void monitor_jobs()
{
    for (int i = 0; i < job_count; i++)
    {
        int status;

        pid_t result = waitpid(
            jobs[i].pid,
            &status,
            WNOHANG
        );

        if (result > 0)
    }
}

```

```
        if (WIFEXITED(status))
        {
            strcpy(jobs[i].status, "Completed");
        }
        else
        {
            strcpy(jobs[i].status, "Terminated");
        }
    }
}

/* Display background jobs */
void show_jobs()
{
    monitor_jobs();

    printf("\nBackground Jobs:\n");
    printf("-----\n");
    printf("Job ID\tPID\tStatus\t\tCommand\n");
    printf("-----\n");

    if (job_count == 0)
    {
        printf("No background jobs.\n");
    }
    else
    {
        for (int i = 0; i < job_count; i++)
        {
            printf("[%d]\t%d\t%-10s\t%s\n",
                    jobs[i].job_id,
                    jobs[i].pid,
                    jobs[i].status,
                    jobs[i].command);
        }

        printf("-----\n");
    }
}

int main()
{
    char input[100];

    printf("Session 15 - Background Jobs\n");
    printf("Commands: run, jobs, exit\n\n");

    while (1)
    {
        /* Monitor jobs before showing prompt */
        monitor_jobs();
```

```
monitor_jobs();

printf("myshell> ");
fflush(stdout);

if (fgets(input, sizeof(input), stdin) == NULL)
    break;

input[strcspn(input, "\n")] = '\0';

/* Launch background process */
if (strcmp(input, "run") == 0)
{
    pid_t pid = fork();

    if (pid < 0)
    {
        perror("fork failed");
    }
    else if (pid == 0)
    {
        printf("\nBackground process started. PID: %d\n",
            getpid());

        /* Simulate a long-running task */
        sleep(10);

        printf("\nBackground process completed. PID: %d\n",
            getpid());

        exit(0);
    }
    else
    {
        add_job(pid, "sleep 10");

        printf("Background job started: [%d] PID %d\n",
            job_count, pid);

        printf("Prompt returned immediately.\n");
    }
}

/* Display jobs */
else if (strcmp(input, "jobs") == 0)
{
    show_jobs();
}

/* Exit */
else if (strcmp(input, "exit") == 0)
{

```

```

        else if (strcmp(input, "exit") == 0)
        {
            printf("\nWaiting for background jobs...\n");

            for (int i = 0; i < job_count; i++)
            {
                if (strcmp(jobs[i].status, "Running") == 0)
                {
                    waitpid(jobs[i].pid, NULL, 0);
                    strcpy(jobs[i].status, "Completed");
                }
            }

            printf("All background jobs completed.\n");
            printf("Exiting shell...\n");

            break;
        }

        else if (strlen(input) == 0)
        {
            continue;
        }

        else
        {
            printf("Invalid command: %s\n",
                input);
            printf("Use: run, jobs, or exit\n");
        }
    }

    return 0;
}

```

Task 2: To Create Job Table, Store Process Groups, Track States, Update Status, Remove Completed Jobs, Validate Job Records

```
lenovo@Manya:~$ nano session16_task2.c
gcc session16_task2.c -o session16_task2
./session16_task2
```

Session 16 - Job Table Management

Commands: run, jobs, update, remove, validate, exit

```
myshell> run
```

Job created: [1]

PID: 2595

PGID: 2595

```
myshell>
```

Child process started.

PID: 2595

PGID: 2595

```
jobs
```

Job Table:

Job ID	PID	PGID	State	Command
[1]	2595	2595	Running	sleep 8

```
myshell> vaChild process completed.
```

```
validate
```

All job records are valid.

```
myshell> update
```

Job states updated successfully.

Job Table:

Job ID	PID	PGID	State	Command
[1]	2595	2595	Completed	sleep 8

```
myshell> jobs
```

Job Table:

Job ID	PID	PGID	State	Command
[1]	2595	2595	Completed	sleep 8

```
myshell> remove
```

Removing Job [1] from job table.

Job table cleanup completed.

```

GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <signal.h>

#define MAX_JOBS 10

typedef struct
{
    int job_id;
    pid_t pid;
    pid_t pgid;
    char command[100];
    char state[20];
} Job;

Job job_table[MAX_JOBS];
int job_count = 0;

/* Create and store a job record */
void add_job(pid_t pid, pid_t pgid, const char *command)
{
    if (job_count >= MAX_JOBS)
    {
        printf("Job table is full.\n");
        return;
    }

    job_table[job_count].job_id = job_count + 1;
    job_table[job_count].pid = pid;
    job_table[job_count].pgid = pgid;

    strcpy(job_table[job_count].command, command);
    strcpy(job_table[job_count].state, "Running");

    job_count++;
}

/* Update the state of jobs */
void update_jobs()
{
    for (int i = 0; i < job_count; i++)
    {
        int status;

        pid_t result = waitpid(
            job_table[i].pid,
            &status,
            WNOHANG

```

```

GNU nano 8.7.1
    );

    if (result > 0)
    {
        if (WIFEXITED(status))
        {
            strcpy(job_table[i].state, "Completed");
        }
        else if (WIFSIGNALED(status))
        {
            strcpy(job_table[i].state, "Terminated");
        }
    }
}

/* Remove completed jobs */
void remove_completed_jobs()
{
    int i = 0;

    while (i < job_count)
    {
        if (strcmp(job_table[i].state, "Completed") == 0 ||
            strcmp(job_table[i].state, "Terminated") == 0)
        {
            printf("Removing Job [%d] from job table.\n",
                job_table[i].job_id);

            for (int j = i; j < job_count - 1; j++)
            {
                job_table[j] = job_table[j + 1];
            }

            job_count--;
        }
        else
        {
            i++;
        }
    }
}

/* Display and validate job records */
void show_jobs()
{
    update_jobs();

    printf("\nJob Table:\n");
    printf("-----\n");
    printf("Job  ID\tPID\tPGID\tState\t\tCommand\n");
    printf("-----\n");
}

```

```

    if (job_count == 0)
    {
        printf("No jobs in the job table.\n");
    }

    for (int i = 0; i < job_count; i++)
    {
        printf("[%d]\t%d\t%d\t%-10s\t%s\n",
            job_table[i].job_id,
            job_table[i].pid,
            job_table[i].pgid,
            job_table[i].state,
            job_table[i].command);
    }

    printf("-----\n");
}

/* Validate job records */
void validate_jobs()
{
    int valid = 1;

    for (int i = 0; i < job_count; i++)
    {
        if (job_table[i].pid <= 0 ||
            job_table[i].pgid <= 0 ||
            strlen(job_table[i].command) == 0)
        {
            valid = 0;
            printf("Invalid job record: [%d]\n",
                job_table[i].job_id);
        }
    }

    if (valid)
    {
        printf("All job records are valid.\n");
    }
}

int main()
{
    char input[100];

    printf("Session 16 - Job Table Management\n");
    printf("Commands: run, jobs, update, remove, validate, exit\n\n");

    while (1)
    {
        printf("myshell> ");

```

```

printf("myshell> ");
fflush(stdout);

if (fgets(input, sizeof(input), stdin) == NULL)
    break;

input[strcspn(input, "\n")] = '\0';

/* Create a new process group/job */
if (strcmp(input, "run") == 0)
{
    pid_t pid = fork();

    if (pid < 0)
    {
        perror("fork failed");
        continue;
    }

    if (pid == 0)
    {
        /* Create a new process group */
        setpgid(0, 0);

        printf("\nChild process started.\n");
        printf("PID: %d\n", getpid());
        printf("PGID: %d\n", getpgrp());

        sleep(8);

        printf("Child process completed.\n");

        exit(0);
    }
    else
    {
        /* Set child process group */
        setpgid(pid, pid);

        add_job(pid, pid, "sleep 8");

        printf("Job created: [%d]\n", job_count);
        printf("PID: %d\n", pid);
        printf("PGID: %d\n", pid);
    }
}

/* Display jobs */
else if (strcmp(input, "jobs") == 0)
{
    show_jobs();
}

```

```

else if (strcmp(input, "update") == 0)
{
    update_jobs();

    printf("Job states updated successfully.\n");

    show_jobs();
}

/* Remove completed jobs */
else if (strcmp(input, "remove") == 0)
{
    update_jobs();
    remove_completed_jobs();

    printf("Job table cleanup completed.\n");
}

/* Validate records */
else if (strcmp(input, "validate") == 0)
{
    validate_jobs();
}

/* Exit */
else if (strcmp(input, "exit") == 0)
{
    printf("\nCleaning up jobs...\n");

    for (int i = 0; i < job_count; i++)
    {
        if (strcmp(job_table[i].state, "Running") == 0)
        {
            kill(job_table[i].pid, SIGTERM);
            waitpid(job_table[i].pid, NULL, 0);
        }
    }

    printf("All resources cleaned up.\n");
    printf("Exiting...\n");

    break;
}

else if (strlen(input) == 0)
{
    continue;
}

else
{
    printf("Invalid command.\n");
}

```

```

/* Exit */
else if (strcmp(input, "exit") == 0)
{
    printf("\nCleaning up jobs...\n");

    for (int i = 0; i < job_count; i++)
    {
        if (strcmp(job_table[i].state, "Running") == 0)
        {
            kill(job_table[i].pid, SIGTERM);
            waitpid(job_table[i].pid, NULL, 0);
        }
    }

    printf("All resources cleaned up.\n");
    printf("Exiting...\n");

    break;
}

else if (strlen(input) == 0)
{
    continue;
}

else
{
    printf("Invalid command.\n");
    printf("Use: run, jobs, update, remove, validate, exit\n");
}
}

return 0;
}

```